

Niveau

Script Intermédiaire

Objectif 3 fichiers Shell

Génération & Archivage de logs

Contexte du TP

Tu travailles comme **administrateur système**. Ton équipe génère des logs applicatifs que tu dois **organiser, filtrer et archiver automatiquement** via des scripts Shell. Ce TP te permettra de maîtriser la création de fichiers, la manipulation de chaînes de caractères, les boucles, les tableaux Bash et la gestion des permissions sous Linux.

Ce TP est composé de 3 scripts Shell : *generate_logs.sh* (création), *process_logs.sh* (réorganisation), et *run_all.sh* (orchestration automatique).

Script 1 — generate_logs.sh

Rôle du script

Ce script **génère des fichiers de log simulés** pour un service applicatif donné dans un environnement précis. Il crée les fichiers, les nomme avec un horodatage précis, et les remplit de lignes de log réalistes avec des niveaux aléatoires.

Paramètres attendus

Paramètre	Variable	Exemple	Description
\$1	SERVICE	nginx	Nom de l'application
\$2	ENV	prod	Environnement cible
\$3	N	4	Nombre de fichiers à créer
\$4	S	6	Nombre de lignes par fichier

Comportement attendu

1. Crée le dossier Logs_ENV (ex: Logs_prod) avec mkdir -p
2. Génère N fichiers nommés : service-YYYY-MM-DD-HH-MM-SS-ms.log
3. Remplit chaque fichier avec S lignes au format [DATE] [LEVEL] service : message
4. Le LEVEL est choisi aléatoirement parmi INFO, WARN, ERROR

5. Attend 300ms entre chaque création de fichier

Exemple de fichier généré

Nom du fichier : **nginx-2024-01-15-10-23-45-123.log**

```
[2024-01-15 10:23:45] [INFO]  nginx : Service started successfully

[2024-01-15 10:23:45] [WARN]  nginx : High memory usage detected

[2024-01-15 10:23:45] [ERROR] nginx : Connection timeout on port 80

[2024-01-15 10:23:45] [INFO]  nginx : Cache refreshed

[2024-01-15 10:23:45] [INFO]  nginx : Connection established

[2024-01-15 10:23:45] [WARN]  nginx : Disk usage above 80 percent
```

Code complet — generate_logs.sh

```
#!/bin/bash

# --- Paramètres ---

SERVICE=$1      # ex: nginx

ENV=$2           # ex: prod

N=$3             # nombre de fichiers

S=$4             # nombre de lignes par fichier

# --- Vérification : on s'assure qu'il y a bien 4 arguments ---

if [ $# -ne 4 ]; then

    echo "Usage: $0 <service> <env> <N> <S>"

    exit 1

fi
```

```
# --- Création du dossier Logs_ENV ---

DOSSIER="Logs_${ENV}"

mkdir -p "$DOSSIER"


# --- Tableaux des niveaux et messages possibles ---

LEVELS=("INFO" "WARN" "ERROR")


MESSAGES_INFO=(

    "Service started successfully"

    "Connection established"

    "Request processed in 120ms"

    "Cache refreshed"

)

MESSAGES_WARN=(

    "High memory usage detected"

    "Slow query detected"

    "Retry attempt 2 of 3"

    "Disk usage above 80 percent"

)

MESSAGES_ERROR=(

    "Connection timeout on port 80"

    "Database unreachable"

    "Authentication failed"

    "Segmentation fault detected"
```

```

)

# --- Boucle : créer N fichiers ---

for i in $(seq 1 $N); do

    # Capturer le timestamp exact au moment de création

    TIMESTAMP=$(date +"%Y-%m-%d-%H-%M-%S-%3N")

    NOM="${SERVICE}-${TIMESTAMP}.log"

    CHEMIN="${DOSSIER}/${NOM}"

    touch "$CHEMIN"

    # Remplir le fichier avec S lignes

    for j in $(seq 1 $S); do

        # Choisir un niveau aléatoire (RANDOM % 3 donne 0, 1 ou 2)

        INDEX=$((RANDOM % 3))

        LEVEL=${LEVELS[$INDEX]}

        DATE_LOG=$(date +"%Y-%m-%d %H:%M:%S")

        # Choisir un message selon le niveau

        if [ "$LEVEL" = "INFO" ]; then

            MSG_INDEX=$((RANDOM % ${#MESSAGES_INFO[@]}))

            MSG=${MESSAGES_INFO[$MSG_INDEX]}

            elif [ "$LEVEL" = "WARN" ]; then

```

```

MSG_INDEX=$((RANDOM % ${#MESSAGES_WARN[@]}))

MSG=${MESSAGES_WARN[$MSG_INDEX]}

else

MSG_INDEX=$((RANDOM % ${#MESSAGES_ERROR[@]}))

MSG=${MESSAGES_ERROR[$MSG_INDEX]}

fi

echo "[${DATE_LOG}] [${LEVEL}] ${SERVICE} : ${MSG}" >> "$CHEMIN"

done

echo "Créé : $CHEMIN"

sleep $(echo "scale=3; 300/1000" | bc) # attendre 300ms

done

echo "✓ $N fichiers log créés dans $DOSSIER"

```

Explications clés du script 1

```
$( (RANDOM % 3) )
```

RANDOM est une variable Bash qui génère un nombre aléatoire. Le modulo 3 donne toujours 0, 1 ou 2 — ce qui correspond aux 3 indices du tableau LEVELS.

```
LEVELS=(...) et  
${LEVELS[$INDEX]}
```

En Bash, on peut créer des tableaux avec (). On accède à un élément avec \${tableau[indice]}.

```
${#MESSAGES_INFO[@]}
```

Donne la taille du tableau MESSAGES_INFO. Utile pour faire un RANDOM % taille et ne jamais dépasser l'indice max.

```
sleep $(echo  
"scale=3; 300/1000" |  
bc)
```

sleep attend en secondes. On divise 300ms par 1000 grâce à bc (calculatrice Bash qui gère les décimales). Résultat : sleep 0.300

```
>> "$CHEMIN"
```

> écrase le fichier, >> ajoute à la fin. Ici on utilise >> pour accumuler les S
lignes sans écraser les précédentes.

Script 2 — process_logs.sh

Rôle du script

Ce script **réorganise les fichiers générés par le script 1** dans une arborescence hiérarchique claire. Il renomme, enrichit le contenu, compte les niveaux de log, et sécurise les fichiers avec les bonnes permissions.

Arborescence créée

```
Archive/

■■■ Logs_prod/

    ■■■ nginx/

        ■■■ 2024/

            ■■■ 01/

                ■■■ 15/

                    ■■■ 102345123.log    ← HH MM SS ms collés

                    ■■■ 104512087.log
```

Nouveau nom du fichier

Le fichier nginx-2024-01-15-10-23-45-123.log devient 102345123.log Explication : HH=10, MM=23, SS=45, ms=123 → 10 + 23 + 45 + 123 = 102345123

Contenu du fichier après traitement

```
SOURCE: nginx-2024-01-15-10-23-45-123.log

PATH: /home/rafik/TpLogs/Logs_prod/nginx-2024-01-15-10-23-45-123.log

ENV: prod

GENERATOR: generate_logs.sh

---

[2024-01-15 10:23:45] [INFO]  nginx : Service started successfully

[2024-01-15 10:23:45] [WARN]  nginx : High memory usage detected
```

```
[2024-01-15 10:23:45] [ERROR] nginx : Connection timeout on port 80

[2024-01-15 10:23:45] [INFO]  nginx : Cache refreshed

[2024-01-15 10:23:45] [INFO]  nginx : Connection established

[2024-01-15 10:23:45] [WARN]  nginx : Disk usage above 80 percent

--- SUMMARY ---

INFO: 3

WARN: 2

ERROR: 1

TOTAL: 6
```

Permissions après traitement

chmod	Propriétaire	Groupe	Autres	Description
440	r (lecture)	r (lecture)	--- (rien)	Prop + groupe lisent, autres exclus
400	r (lecture)	--- (rien)	--- (rien)	Seul le propriétaire lit
600	rw (lect+écrit)	--- (rien)	--- (rien)	Propriétaire lit et écrit

Code complet — process_logs.sh

```
#!/bin/bash

# --- Paramètre : dossier à traiter ---

DOSSIER=$1

# --- Vérifications ---

if [ $# -ne 1 ]; then

    echo "Usage: $0 <Logs_env>"

    exit 1
```



```
fi

if [ ! -d "$DOSSIER" ]; then

    echo "Erreur : le dossier '$DOSSIER' n'existe pas."

    exit 1

fi

# --- Extraire l'environnement (Logs_prod → prod) ---

ENV=$(echo "$DOSSIER" | cut -d'_' -f2)

# --- Créer le dossier Archive ---

mkdir -p "Archive"

# --- Parcourir tous les fichiers .log ---

for FILEPATH in "${DOSSIER}"/*.log; do

    [ -f "$FILEPATH" ] || continue    # sécurité si dossier vide

    FILENAME=$(basename "$FILEPATH")

    SANS_EXT="${FILENAME%.log}"

    # Extraire le service (avant le premier "-")

    SERVICE=$(echo "$SANS_EXT" | cut -d'-' -f1)
```

```
# Extraire les composantes de la date

DATE_PART=$(echo "$SANS_EXT" | cut -d'-' -f2-)

ANNEE=$(echo "$DATE_PART" | cut -d'-' -f1) # 2024

MOIS=$(echo "$DATE_PART" | cut -d'-' -f2) # 01

JOUR=$(echo "$DATE_PART" | cut -d'-' -f3) # 15

HEURE=$(echo "$DATE_PART" | cut -d'-' -f4) # 10

MINUTE=$(echo "$DATE_PART" | cut -d'-' -f5) # 23

SECONDE=$(echo "$DATE_PART" | cut -d'-' -f6) # 45

MS=$(echo "$DATE_PART" | cut -d'-' -f7) # 123

# Nouveau nom : HHMMSSms.log

NOUVEAU_NOM="${HEURE}${MINUTE}${SECONDE}${MS}.log"

# Dossier destination dans Archive/

DEST_DIR="Archive/${DOSSIER}/${SERVICE}/${ANNEE}/${MOIS}/${JOUR}"

mkdir -p "$DEST_DIR"

# Chemin absolu de l'ancien fichier

ANCIEN_CHEMIN=$(realpath "$FILEPATH")

# Lire le contenu original

CONTENU=$(cat "$FILEPATH")
```

```
# Compter les niveaux de log

NB_INFO=$(echo "$CONTENU" | grep -c "\[INFO\)")

NB_WARN=$(echo "$CONTENU" | grep -c "\[WARN\)")

NB_ERROR=$(echo "$CONTENU" | grep -c "\[ERROR\)")

NB_TOTAL=$((NB_INFO + NB_WARN + NB_ERROR))


# Déplacer le fichier vers sa destination

mv "$FILEPATH" "${DEST_DIR}/${NOUVEAU_NOM}"


# Réécrire le contenu enrichi (4 lignes entête + contenu + résumé)

{

    echo "SOURCE: ${FILENAME}"

    echo "PATH: ${ANCIEN_CHEMIN}"

    echo "ENV: ${ENV}"

    echo "GENERATOR: generate_logs.sh"

    echo "---"

    echo "$CONTENU"

    echo "--- SUMMARY ---"

    echo "INFO: ${NB_INFO}"

    echo "WARN: ${NB_WARN}"

    echo "ERROR: ${NB_ERROR}"

    echo "TOTAL: ${NB_TOTAL}"

} > "${DEST_DIR}/${NOUVEAU_NOM}"
```

```

# Permissions : prop + groupe lisent, autres rien

chmod 440 "${DEST_DIR}/${NOUVEAU_NOM}"

echo "✓ ${FILENAME} → ${DEST_DIR}/${NOUVEAU_NOM}"

done

echo ""

echo "✓ Traitement de '$DOSSIER' terminé dans Archive/"

```

Explications clés du script 2

```
cut -d'-' -f2-
```

Coupe la chaîne sur le délimiteur '-'. -f2- signifie 'du 2ème champ jusqu'à la fin'. Ex: nginx-2024-01-15 → 2024-01-15

```
[ ! -d "$DOSSIER" ]
```

Vérifie que \$DOSSIER n'est PAS un dossier existant. != négation, -d = est un dossier.

```
[ -f "$FILEPATH" ] ||
continue
```

Si \$FILEPATH n'est pas un fichier (ex: glob vide), on saute l'itération avec continue. Sécurité importante.

```
CONTENU=$(cat
"$FILEPATH")
```

Lit tout le contenu du fichier et le stocke dans la variable CONTENU. Utile pour compter les niveaux avant de déplacer le fichier.

```
grep -c "[INFO\]"
```

grep -c compte le nombre de lignes qui contiennent le motif. Les \ échappent les crochets qui ont un sens spécial en regex.

```
{ echo ...; echo ...; } >
fichier
```

Les accolades groupent plusieurs commandes. Tout ce qu'elles affichent est redirigé vers le fichier avec >. Plus propre que plusieurs >>.

```
realpath "$FILEPATH"
```

Retourne le chemin absolu complet du fichier. Ex: ./Logs_prod/nginx.log → /home/rafik/TpLogs/Logs_prod/nginx.log

Script 3 — run_all.sh (orchestration)

Rôle du script

Ce script **lance automatiquement** le script 1 (9 fois) puis le script 2 (3 fois), et vérifie le résultat final. C'est le **chef d'orchestre** : au lieu de taper 12 commandes à la main, une seule suffit.

■ Les scripts 1 et 2 ne sont PAS modifiés. run_all.sh se contente de les appeler dans le bon ordre avec les bons arguments.

Code complet — run_all.sh

```
#!/bin/bash

# =====

# Script principal : lance tout le TP en 1 commande

# =====

SCRIPT_GENERATION="./generate_logs.sh"

SCRIPT_PROCESS="./process_logs.sh"

N=4      # nombre de fichiers par service

S=6      # nombre de lignes par fichier

# --- Vérifier que les 2 scripts existent ---

if [ ! -f "$SCRIPT_GENERATION" ]; then

    echo "Erreur : generate_logs.sh introuvable"

    exit 1

fi

if [ ! -f "$SCRIPT_PROCESS" ]; then
```

```
    echo "Erreur : process_logs.sh introuvable"

    exit 1

fi

# =====

echo "===== "

echo "    ÉTAPE 1 : Génération des logs"

echo "===== "


echo "--- Logs_prod ---"

$SCRIPT_GENERATION nginx  prod $N $S

$SCRIPT_GENERATION mysql  prod $N $S

$SCRIPT_GENERATION apache prod $N $S


echo "--- Logs_staging ---"

$SCRIPT_GENERATION nginx  staging $N $S

$SCRIPT_GENERATION mysql  staging $N $S

$SCRIPT_GENERATION apache staging $N $S


echo "--- Logs_dev ---"

$SCRIPT_GENERATION nginx  dev $N $S

$SCRIPT_GENERATION mysql  dev $N $S

$SCRIPT_GENERATION apache dev $N $S
```

```
# =====

echo ""

echo "=====

echo "    ÉTAPE 2 : Traitement et archivage"

echo "=====

$SCRIPT_PROCESS Logs_prod

$SCRIPT_PROCESS Logs_staging

$SCRIPT_PROCESS Logs_dev

# =====

echo ""

echo "=====

echo "    ÉTAPE 3 : Vérification finale"

echo "=====

echo "--- Arborescence Archive/ ---"

find Archive/ -type f

echo ""

echo "--- Permissions des fichiers ---"

find Archive/ -type f -name "*.log" | head -5 | xargs ls -l

echo ""
```

```

echo "--- Contenu d'un fichier exemple ---"

EXEMPLE=$(find Archive/ -type f -name "*.log" | head -1)

echo "Fichier : $EXEMPLE"

cat "$EXEMPLE"


echo ""

echo "✓ TP terminé avec succès !"

```

Explications clés du script 3

```
SCRIPT_GENERATION="./generate_logs.sh"
```

On stocke le nom du script dans une variable. Si tu renommes le fichier, tu changes juste cette ligne au lieu de chercher partout dans le code.

```
if [ ! -f
"$SCRIPT_GENERATION" ]
```

Vérifie que le script existe avant de lancer. ! = NON, -f = est un fichier. Évite un plantage obscur au milieu de l'exécution.

```
$SCRIPT_GENERATION nginx prod
$N $S
```

Appelle le script avec ses 4 arguments. \$N et \$S sont remplacés automatiquement par leurs valeurs (4 et 6).

```
find Archive/ -type f | head
-5 | xargs ls -l
```

Chaîne de pipes : find liste les fichiers → head -5 garde les 5 premiers → xargs ls -l affiche les détails (dont les permissions) de chacun.

Démarche complète — du début à la fin

1 Créer le dossier de travail

```
mkdir -p ~/TpLogs && cd ~/TpLogs
```

2 Créer les 3 scripts vides

```
touch generate_logs.sh process_logs.sh run_all.sh
```

3 Écrire les scripts avec nano

```
nano generate_logs.sh
```

```
nano process_logs.sh
```

```
nano run_all.sh
```

4 Rendre les scripts exécutable

```
chmod +x generate_logs.sh process_logs.sh run_all.sh
```

5 Option A — Tout lancer en une commande

```
./run_all.sh
```

6 Option B — Lancer manuellement le script 1 (9 fois)

```
./generate_logs.sh nginx prod 4 6  
  
./generate_logs.sh mysql prod 4 6  
  
./generate_logs.sh apache prod 4 6  
  
./generate_logs.sh nginx staging 4 6  
  
./generate_logs.sh mysql staging 4 6  
  
./generate_logs.sh apache staging 4 6  
  
./generate_logs.sh nginx dev 4 6  
  
./generate_logs.sh mysql dev 4 6  
  
./generate_logs.sh apache dev 4 6
```

7 Option B — Lancer manuellement le script 2 (3 fois)

```
./process_logs.sh Logs_prod
```

```
./process_logs.sh Logs_staging
```

```
./process_logs.sh Logs_dev
```

8 Vérifier le résultat

```
find Archive/ -type f
```

```
ls -l Archive/Logs_prod/nginx/2024/01/15/
```

```
cat $(find Archive/ -type f | head -1)
```

Récapitulatif des commandes utilisées

Commande	Script(s)	Rôle
<code>mkdir -p</code>	1, 2, 3	Crée un dossier et ses parents si inexistants
<code>touch</code>	1	Crée un fichier vide
<code>date +"format"</code>	1	Récupère la date/heure système formatée
<code>sleep 0.300</code>	1	Attend 300ms entre chaque fichier
<code>for i in \$(seq)</code>	1, 2	Boucle sur une plage de nombres
<code>RANDOM % N</code>	1	Génère un entier aléatoire entre 0 et N-1
<code>echo ... >> file</code>	1	Ajoute une ligne à la fin d'un fichier
<code>basename</code>	2	Extrait le nom de fichier sans le chemin
<code>cut -d'-' -f2-</code>	2	Découpe une chaîne sur un délimiteur
<code>realpath</code>	2	Retourne le chemin absolu d'un fichier
<code>grep -c</code>	2	Compte le nombre de lignes matchant un motif
<code>mv</code>	2	Déplace (et renomme) un fichier
<code>{ } > fichier</code>	2	Groupe des echos et redirige vers un fichier
<code>chmod 440</code>	2	Définit les permissions du fichier
<code>find -type f</code>	3	Liste tous les fichiers d'une arborescence
<code>head -N</code>	3	Garde seulement les N premières lignes
<code>xargs ls -l</code>	3	Passe chaque ligne comme argument à ls -l
<code>\$#</code>	1, 2	Nombre d'arguments passés au script
<code>\$0</code>	1, 2, 3	Nom du script lui-même
<code>exit 1</code>	1, 2, 3	Quitte le script avec un code d'erreur

Résultat attendu

Structure finale du dossier TpLogs/

TpLogs/

■■■ generate_logs.sh ← script 1

■■■ process_logs.sh ← script 2

■■■ run_all.sh ← script 3

■■■ Logs_prod/ ← vidé après process_logs.sh

```
■■■ Logs_staging/

■■■ Logs_dev/

■■■ Archive/

    ■■■ Logs_prod/

        ■ ■■■ nginx/2024/01/15/

            ■ ■ ■■■ 102345123.log

            ■ ■ ■■■ 104512087.log

        ■ ■■■ mysql/...

        ■ ■■■ apache/...

    ■■■ Logs_staging/

    ■ ■■■ ...

    ■■■ Logs_dev/

    ■■■ ...
```

Vérification des permissions

```
$ ls -l Archive/Logs_prod/nginx/2024/01/15/

-r--r----- 1 rafik rafik 312 Jan 15 10:23 102345123.log

-r--r----- 1 rafik rafik 298 Jan 15 10:24 102712045.log


# -r--r----- = chmod 440

# r-- = propriétaire peut lire

# r-- = groupe peut lire

# --- = autres ne peuvent rien faire
```

■ Si tu vois -r----- c'est chmod 400 (seul le propriétaire lit). Pour ce TP on utilise chmod 440 (propriétaire ET groupe lisent).

Bonne chance ! ■